CLASE 9

La imagen es una matriz

OpenCV · Leer, recortar, transformar y dibujar

Arranca el último tramo con contenido nuevo: visión por computadora.

DÓNDE ESTAMOS

El Obligatorio 1 está entregado. Hoy arranca el último tramo con contenido nuevo.

Son tres clases: hoy, el 23/10 y el 30/10. Después, todo el tiempo de clase es para el Obligatorio 2.

ARRANQUE

Lo que queda del curso

Ocho viernes. Los tres primeros enseñan; los otros cinco son para el Obligatorio 2.

Clase	Fecha	Qué pasa
9	16/10	La imagen como dato: abrir, recortar, transformar, dibujar
10	23/10	La cámara en vivo: el bucle de captura y los filtros
11	30/10	Que el programa reaccione a lo que ve: color, caras y manos
12	6/11	Se entrega la letra del Obligatorio 2
13 y 14	13/11 y 20/11	Práctico del Obligatorio 2
15	27/11	Entrega del Obligatorio 2
16	4/12	Defensa del Obligatorio 2

La clase 12 también es de trabajo: la letra se entrega al principio y el resto es práctico.

EL OBJETIVO DEL TRAMO

Convertir lo que ve la cámara en una coordenada (x, y), y usarla en un programa de pygame.

Todo lo de estas tres clases sirve para llegar a esa frase. Hoy empezamos por el principio: qué es una imagen para un programa.

ARRANQUE

El Obligatorio 2, en tres datos

Lo que conviene saber desde hoy, aunque la letra llegue el 6/11.

60 puntos

Sobre 100. El Obligatorio 1 valió los otros 40.

Individual

Cada uno entrega su propio programa, y responde por él.

Con defensa

El 4/12 cada uno explica su código, en persona.

La defensa es la razón por la que estas tres clases terminan con preguntas.

ARRANQUE

Qué es la defensa

Una conversación corta, el 4 de diciembre, sobre tu propio programa.

- Te voy a pedir que expliques partes de tu código: qué hace una línea, por qué está ahí, qué pasa si la sacás.

- No se evalúa quién escribió el código. Se evalúa si lo entendés.
- Como es individual, nadie responde por vos.
- Cada clase termina con cuatro preguntas. Son del mismo tipo que las de ese día.

LA IA SE PUEDE USAR AL 100 %

Con la misma condición de siempre: entender qué hace cada parte del código, o comentarlo a mano. La defensa es donde esa condición se pone a prueba.

ARRANQUE

Cómo cambian las clases desde hoy

Cuatro cambios respecto de las clases anteriores.

- No hay ejercicios domiciliarios: todo se hace acá, en clase.
- Menos tiempo de exposición y más de trabajo: 80 minutos de teórico y 70 de práctico.
- Lo que no sale, se pregunta en el momento. No hay noche para pelearse solo con un error.
- Los ejercicios van en orden de prioridad: el primero no se saltea; el último puede quedar a medias.

DÓNDE VA EL CÓDIGO

En el repositorio ejercicios-clase, carpeta clase-09. Un archivo .py por ejercicio, un commit por ejercicio, y push antes de irse.

¿Qué vamos a hacer hoy?

01 El stack: instalar y fijar versiones

02 La imagen como dato: shape, píxel, BGR

03 Recortar, y cuando el archivo no está

04 Transformar: gris, tamaño, espejo

05 Dibujar encima, y pegar una imagen en otra

06 Una carpeta entera de un saque

07 Práctico: el procesador de obras

01

El stack

Quince minutos ahora, o tres clases rotas después

QUÉ ES UNA BIBLIOTECA

Una biblioteca es código que escribió otra persona, listo para usar con import.

pygame fue la primera del curso. Hoy se suman dos más. Ninguna viene con Python: hay que instalarlas.

01 · LAS TRES

Las tres bibliotecas del tramo

Se instalan las tres hoy, aunque MediaPipe recién se usa en la clase 11.

OpenCV

Abre, transforma y analiza imágenes y video. Es la de hoy y la de la clase 10.

MediaPipe

Detecta manos, caras y cuerpos en una imagen. Llega en la clase 11.

pygame

La ventana, el bucle y la interacción. Ya la conocen de la clase 5.

Se instalan hoy para detectar problemas ahora, con tiempo, y no el 30/10.

01 · OPENCV

Qué es OpenCV

Open Source Computer Vision: visión por computadora, de código abierto.

- Es la biblioteca de visión por computadora más usada del mundo.
- Se usa en cámaras de seguridad, en autos, en filtros de redes sociales y en instalaciones artísticas.

- Se instala con el nombre opencv-contrib-python.
- Pero en el código se importa como cv2. El nombre del paquete y el del import no coinciden, y es normal.

POR QUÉ CV2

Viene de la versión 2 de OpenCV, que cambió la forma de usarlo desde Python. La anterior se importaba como cv. Hoy van por la versión 4, pero el nombre quedó.

01 · VERSIONES

Por qué hay que elegir la versión

Hasta ahora instalábamos sin pensar en la versión. Desde hoy, no.

- pip install sin aclarar versión baja siempre la última.
- Las últimas versiones de OpenCV y de MediaPipe sacaron cosas que vamos a necesitar.
- Y además no se llevan entre sí: cada una pide una versión distinta de numpy.
- numpy es la biblioteca que guarda las imágenes como tablas de números. Se instala sola, con las otras.

LA LECCIÓN GENERAL

En un proyecto real, las versiones se fijan. Si no se fijan, el programa que hoy anda deja de andar el día que alguien lo instala de nuevo, sin haber cambiado una sola línea.

01 · POR QUÉ

Qué versión usamos de cada una

Las versiones nuevas sacaron cosas que necesitamos. Por eso no usamos la última.

Paquete	La última	Qué le falta	La que usamos
`opencv-contrib-python`	5.0.0	los detectores de caras	4.11.0.86
`mediapipe`	1.1.1	la forma simple + los modelos	0.10.21
`pygame`	2.6.1	nada: va la última	2.6.1

Y no sirve cualquier OpenCV 4: desde la 4.12 choca con MediaPipe por la versión de numpy. La 4.11 es la última que convive con MediaPipe.

ANTES DE SEGUIR

Las versiones nuevas no borraron nada. Cambiaron cómo se instala y cómo se escribe el código.

Todo lo que vamos a usar sigue existiendo en las últimas versiones. Al final de esta sección está, paso a paso, qué hay que hacer si querés usarlas.

01 · POR QUÉ

Por qué el curso usa las versiones anteriores

No es por nostalgia. Son cuatro razones prácticas.

- Todo viene adentro del paquete: los detectores de caras y los modelos de manos. No hay nada que bajar aparte.
- Por eso funcionan sin internet, que en un salón con quince computadoras instalando a la vez importa.
- El código es más corto: la detección de manos son 6 líneas en vez de 13.
- Es el código que aparece en casi todos los tutoriales, y el que sugiere la IA. Con las versiones nuevas, eso no corre.

Y NO VAN A DESAPARECER

Las versiones viejas siguen publicadas y se pueden instalar siempre. El único límite es el Python: MediaPipe 0.10.21 funciona hasta Python 3.12, y no en los que salgan después.

01 · INSTALAR

El archivo requirements.txt

Un archivo de texto con la lista de lo que hay que instalar. Tres líneas.

requirements.txt

```
opencv-contrib-python==4.11.0.86
mediapipe==0.10.21
pygame==2.6.1
```

- Un paquete por línea.
- == fija la versión exacta: ni una más nueva, ni una más vieja.
- Va en la carpeta del repositorio ejercicios-clase, y se commitea como cualquier archivo.
- Es opencv-contrib-python, no opencv-python: es el que pide MediaPipe, y trae todo lo del otro.

01 · INSTALAR

Instalar, en cuatro pasos

Todo en la terminal de VS Code, la misma donde usan git.

1. python --version — para saber qué Python tenés. Tiene que ser 3.9, 3.10, 3.11 o 3.12.
2. Crear requirements.txt en la carpeta ejercicios-clase, con las tres líneas de la diapositiva anterior.
3. pip install -r requirements.txt — la -r significa: leé la lista de este archivo.
4. Correr el programa de verificación de la diapositiva siguiente.

Si pip no existe, probá con pip3. Si tampoco, con python -m pip. Son la misma herramienta con otro nombre.

01 · VERIFICAR

Verificar que quedó todo

Si este programa imprime las tres versiones, el stack está listo.

verificar.py

```
import cv2, mediapipe, pygame

print("OpenCV   ", cv2.__version__)
print("MediaPipe", mediapipe.__version__)
print("pygame   ", pygame.version.ver)
```

SALIDA

```
pygame 2.6.1 (SDL 2.28.4, Python 3.12.3)
Hello from the pygame community.
OpenCV    4.11.0
MediaPipe 0.10.21
pygame    2.6.1
```

- Las dos primeras líneas son el saludo que pygame imprime siempre al importarse. No es un error.
- OpenCV dice 4.11.0, sin el .86: ese último número es del paquete, no de OpenCV.
- Si alguna de las tres líneas no sale, el import de esa biblioteca falló: avisá.

01 · TU PYTHON

Si tenés Python 3.13 o más

MediaPipe 0.10.21 solo existe hasta Python 3.12. De eso depende lo que instalan.

Python 3.9 a 3.12

```
python --version
# Python 3.12.x

pip install -r \
    requirements.txt
```

Listo. Las tres versiones entran y en la clase 11 no hay nada que descargar.

Python 3.13 o más

```
python --version
# Python 3.13.x

# mediapipe==0.10.21
# NO se puede instalar
```

Dos caminos: instalar Python 3.12 además del que tenés, o usar las versiones nuevas. Los dos están al final de esta sección.

Si tenés 3.13, decilo ahora. Descubrirlo el 30/10, en la clase de MediaPipe, es tarde.

01 · SI FALLA

Los errores de instalación más probables

Y qué significa cada uno.

- "command not found: pip" — probá con pip3, y si tampoco, con python -m pip.
- "No matching distribution found for mediapipe==0.10.21" — tenés Python 3.13 o más.
- "No matching distribution found for opencv-contrib-python" en una Mac — tenés macOS 12 o anterior: OpenCV 4.11 pide macOS 13 o más.
- Cualquier otro mensaje: copialo entero y mostralo. El mensaje dice qué pasó.

ANTES DE SEGUIR

Que nadie avance con el stack roto. Todo lo que viene después necesita que import cv2 funcione.

01 · LA IA

La IA no sabe en qué versión estás

La lección más útil de todo el tramo. Conviene aprenderla hoy y no en el Obligatorio.

- Si Copilot sugiere cv2.data.haarcascades y el archivo no existe, estás en OpenCV 5.
- Si sugiere mp.solutions.hands y dice que mediapipe no tiene solutions, estás en MediaPipe 1.x.
- En los dos casos la sugerencia está bien escrita, es plausible, y es incorrecta para tu máquina.
- No está mal tu código: está desactualizada la sugerencia. Mirá el requirements.txt del curso.

POR QUÉ PASA

La IA aprendió de millones de líneas escritas cuando esas versiones eran las normales. No tiene forma de saber qué tenés instalado vos.

01 · LA IA

Cómo se ve una sugerencia desactualizada

El código que sugiere casi cualquier IA para detectar manos, corrido con MediaPipe 1.x.

manos.py

```
import mediapipe as mp

manos = mp.solutions.hands.Hands()
```

SALIDA

```
Traceback (most recent call last):
  File "manos.py", line 3
AttributeError: module
'mediapipe' has no attribute
'solutions'
```

- El código es correcto: con MediaPipe 0.10.21 funciona.
- Con MediaPipe 1.x, solutions ya no existe, y el error lo dice textual.
- Cuando un error dice que algo no existe en una biblioteca, lo primero es sospechar de la versión.

SI QUERÉS USAR LAS ÚLTIMAS

Con las versiones nuevas se puede hacer todo lo del curso. Cuesta dos archivos más y un poco más de código.

Las próximas diapositivas explican qué cambia en cada biblioteca, y qué hacer. Es también el camino para quien tenga Python 3.13 o más.

01 · LAS ÚLTIMAS

El requirements.txt con las versiones nuevas

Las tres últimas versiones se llevan bien entre sí: se probaron instalándolas juntas.

requirements.txt

```
opencv-contrib-python==5.0.0.93
mediapipe==1.1.1
pygame==2.6.1
```

- Funcionan con cualquier Python, incluido el 3.13.
- Instalan numpy 2 por su cuenta; no hay que pedirlo.
- Lo que no traen son los archivos de los modelos: esos se bajan aparte, como se explica a continuación.

01 · OPENCV 5

OpenCV 5: qué cambia

El detector de caras sigue existiendo. Cambiaron dos cosas de cómo se consigue.

- Se mudó: en OpenCV 5, el detector de caras ya no está en opencv-python. Solo en opencv-contrib-python, el que usamos.
- El paquete ya no trae los archivos .xml con los modelos de caras. Hay que bajarlos aparte.
- El archivo se llama haarcascade_frontalface_default.xml, y se guarda en la carpeta del proyecto, al lado del .py.
- En el código cambia una sola línea: la que dice dónde está ese archivo.

DÓNDE ESTÁ EL ARCHIVO

En el repositorio oficial de OpenCV en GitHub (opencv/opencv), carpeta data/haarcascades. Es el mismo archivo que trae la 4.11: el de una versión funciona en la otra.

01 · OPENCV 5

Qué pasa si no hacés nada

La línea de siempre, con OpenCV 5 y sin haber bajado el archivo.

caras.py

```
import cv2

ruta = cv2.data.haarcascades
ruta = ruta + "haarcascade_frontalface_default.xml"
caras = cv2.CascadeClassifier(ruta)

img = cv2.imread("fotos/retrato.png")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
caras.detectMultiScale(gris)
```

SALIDA

```
[ERROR] Can't open file:
'.../cv2/data/haarcascade_
frontalface_default.xml'
Traceback (most recent call last):
cv2.error: (-215:Assertion
failed) !empty() in function
'detectMultiScale'
```

- El primer aviso sale al crear el detector: no encontró el archivo. Pero no corta el programa.
- Corta recién al usarlo: !empty() quiere decir "el detector no puede estar vacío".
- Es el mismo patrón que imread: falla en silencio y el error aparece una línea después.

La línea que cambia

Las dos formas, una abajo de la otra. Se usa una sola, según la versión.

caras.py

```python
import cv2

XML = "haarcascade_frontalface_default.xml"

# OpenCV 4.11: el XML viene adentro del paquete
ruta = cv2.data.haarcascades + XML
caras = cv2.CascadeClassifier(ruta)

# OpenCV 5: el XML lo bajaste vos, y está
# en la carpeta del proyecto
caras = cv2.CascadeClassifier(XML)
```

- cv2.data.haarcascades es la carpeta donde la 4.11 guarda sus archivos .xml.
- En OpenCV 5 esa carpeta existe, pero está vacía.
- Con el archivo al lado del .py, alcanza con su nombre. Ojo con la carpeta desde donde se ejecuta: es el error de las rutas relativas de la sección 03.

MediaPipe 1.x: qué cambia

La detección de manos, caras y cuerpo sigue existiendo. Cambió la forma de usarla.

- mp.solutions, la forma vieja, ya no existe. Ahora todo está en mp.tasks.vision.
- Los modelos no vienen en el paquete: hay que bajar un archivo .task por cada cosa que se quiera detectar.
- Para las manos, el archivo es hand_landmarker.task, y va en la carpeta del proyecto.
- El código es más largo: antes de crear el detector hay que armar sus opciones.

DÓNDE ESTÁ EL MODELO DE MANOS

En la página oficial de MediaPipe, sección Hand landmarker, apartado Models. El enlace directo: storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/latest/hand_landmarker.task

01 · MEDIAPIPE

Manos con MediaPipe 0.10.21: la del curso

Seis líneas, contando el import. El modelo vino adentro del paquete.

manos.py

```python
import mediapipe as mp

manos = mp.solutions.hands.Hands(max_num_hands=1)

resultado = manos.process(frame_rgb)
if resultado.multi_hand_landmarks:
    mano = resultado.multi_hand_landmarks[0]
    punto = mano.landmark[8]
```

- frame_rgb es un cuadro de la cámara pasado a RGB. Lo vemos en la clase 11.
- process analiza el cuadro y devuelve lo que encontró.
- El punto 8 es la punta del dedo índice: cada mano tiene 21 puntos numerados.

01 · MEDIAPIPE 1

Lo mismo con MediaPipe 1.x

Hace exactamente lo mismo que la anterior, en 13 líneas en vez de 6.

manos.py

```
import mediapipe as mp

vision = mp.tasks.vision
opciones = vision.HandLandmarkerOptions(
    base_options=mp.tasks.BaseOptions(
        model_asset_path="hand_landmarker.task"),
    num_hands=1)
manos = vision.HandLandmarker.create_from_options(
    opciones)

imagen = mp.Image(image_format=mp.ImageFormat.SRGB,
                  data=frame_rgb)
resultado = manos.detect(imagen)
if resultado.hand_landmarks:
    punto = resultado.hand_landmarks[0][8]
```

- model_asset_path es la ruta al archivo .task que hay que bajar. Sin él: FileNotFoundError.
- El cuadro hay que envolverlo en un mp.Image antes de pasárselo al detector.
- process pasa a llamarse detect, y multi_hand_landmarks pasa a llamarse hand_landmarks. El punto 8 sigue siendo la punta del índice.

01 · RESUMEN

Qué hacer según la versión

Las tres situaciones posibles, en una tabla.

Si usás	Qué instalar	Qué bajar aparte	Qué cambia en el código
las versiones del curso	`requirements.txt del curso`	`nada`	nada
OpenCV 5	`opencv-contrib-python`	`haarcascade_frontalface_default.xml`	la ruta del .xml
MediaPipe 1.x	`mediapipe`	`hand_landmarker.task`	se usa mp.tasks.vision

Se puede usar una combinación nueva y otra vieja, salvo OpenCV 5 con MediaPipe 0.10.21: piden versiones de numpy que no se llevan.

02

La imagen como dato

Qué ve el programa cuando abre una foto

LA IDEA DE HOY

Para un programa, una imagen es una tabla de números.

No ve caras, ni colores, ni formas. Ve filas y columnas de números. Todo lo de las próximas tres clases sale de esta frase.

02 · LA TABLA

Una imagen de 6 × 4 píxeles, agrandada

Cada cuadrado es un píxel. Los tres números de adentro son su color.

y ↓ \ x →	0	1	2	3	4	5
0	63 255 182	60 70 180	190 120 50	90 170 90	245 245 245	20 22 43
1	0 0 255	0 255 0	255 0 0	0 0 0	255 255 255	0 200 255
2	128 128 128	40 40 40	200 200 0	150 60 200	30 140 240	240 210 180
3	60 70 180	63 255 182	10 90 30	180 180 250	90 40 10	220 220 220

- Tiene 4 filas, numeradas de 0 a 3, y 6 columnas, de 0 a 5.
- Se cuenta desde 0, como los índices de las listas de la clase 3.
- Cada píxel guarda tres números: azul, verde y rojo, en ese orden.
- Una foto de celular de 4000 × 3000 tiene 12 millones de estos cuadrados.

02 · EL PÍXEL

Qué es un píxel

El punto más chico de una imagen. Tiene un solo color.

- Ese color se guarda como tres números: cuánto azul, cuánto verde y cuánto rojo tiene.
- Cada número va de 0 a 255.
- 0 significa nada de ese color; 255, todo lo posible.
- (0, 0, 0) es negro: nada de nada. (255, 255, 255) es blanco: todo de todo.

POR QUÉ 255

Es el número más grande que entra en un byte, la unidad mínima de memoria. Un byte por color, tres bytes por píxel. Son las mismas tuplas de color de la clase 2 y de pygame.

02 · ABRIR

Abrir una imagen: el import

El programa completo tiene cinco líneas de trabajo. Las vemos de a una.

abrir.py

```
import cv2

img = cv2.imread("fotos/retrato.png")

cv2.imshow("la obra", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- import cv2 trae OpenCV al programa.
- Va arriba de todo, como siempre.
- Desde acá, todo lo de OpenCV se escribe cv2.algo: cv2.imread, cv2.imshow.

02 · ABRIR

imread: leer la imagen del disco

imread viene de image read: leer una imagen.

abrir.py

```
import cv2

img = cv2.imread("fotos/retrato.png")

cv2.imshow("la obra", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- Recibe la ruta del archivo, entre comillas, como texto.
- "fotos/retrato.png" significa: el archivo retrato.png, adentro de la carpeta fotos.
- Devuelve la imagen convertida en tabla de números. La guardamos en la variable img.
- Si no encuentra el archivo, no da error: devuelve None. Lo vemos en la sección 03.

02 · ABRIR

imshow: mostrarla en una ventana

imshow viene de image show: mostrar una imagen.

abrir.py

```
import cv2

img = cv2.imread("fotos/retrato.png")

cv2.imshow("la obra", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- El primer argumento es el título de la ventana: el texto que aparece arriba.
- El segundo es la imagen que se va a mostrar.
- Pero con esta línea sola todavía no se ve nada: falta la siguiente.

02 · ABRIR

waitKey: la línea que todos olvidan

waitKey viene de wait key: esperar una tecla.

abrir.py

```
import cv2

img = cv2.imread("fotos/retrato.png")

cv2.imshow("la obra", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- waitKey(0) frena el programa hasta que se apriete cualquier tecla. El 0 significa: esperar para siempre.
- Mientras espera, la ventana se dibuja y responde.
- Sin esta línea, la ventana aparece en blanco, o se abre y se cierra en un instante.
- waitKey(2000) espera 2000 milisegundos, o sea 2 segundos, y sigue sola.

02 · ABRIR

destroyAllWindows: cerrar todo

destroy all windows: destruir todas las ventanas.

abrir.py

```
import cv2

img = cv2.imread("fotos/retrato.png")

cv2.imshow("la obra", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- Cierra todas las ventanas que abrió OpenCV.
- Sin ella, la ventana puede quedar colgada después de que termina el programa.
- Las tres van siempre juntas y en este orden: imshow, waitKey, destroyAllWindows.

02 · ABRIR

Abrir y mostrar, completo

Las cinco líneas juntas. Es el programa más chico que muestra una imagen.

abrir.py

```
import cv2

img = cv2.imread("fotos/retrato.png")

cv2.imshow("la obra", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

SALIDA

```
(se abre una ventana con la
 imagen, y se cierra al
 apretar cualquier tecla)
```

- Hace lo que en pygame hacían event.get() y flip() juntos: atiende la ventana y la muestra.
- La ventana se abre del tamaño real de la imagen: una foto de celular puede no entrar en la pantalla.
- Para cerrarla, una tecla. La cruz de la ventana no alcanza: el programa sigue esperando la tecla.

02 · SHAPE

¿Qué imprime?

retrato.png mide 600 píxeles de ancho por 800 de alto. Treinta segundos.

medidas.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

print(img.shape)
print(img.shape[0], "de alto")
print(img.shape[1], "de ancho")
```

SALIDA

?

- shape significa forma: devuelve las medidas de la imagen.
- La pregunta es en qué orden.
- Pensalo antes de pasar a la siguiente.

02 · SHAPE

El tamaño, y el orden que sorprende

Primero el alto. Después el ancho. Al final, los canales.

medidas.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

print(img.shape)
print(img.shape[0], "de alto")
print(img.shape[1], "de ancho")
```

SALIDA

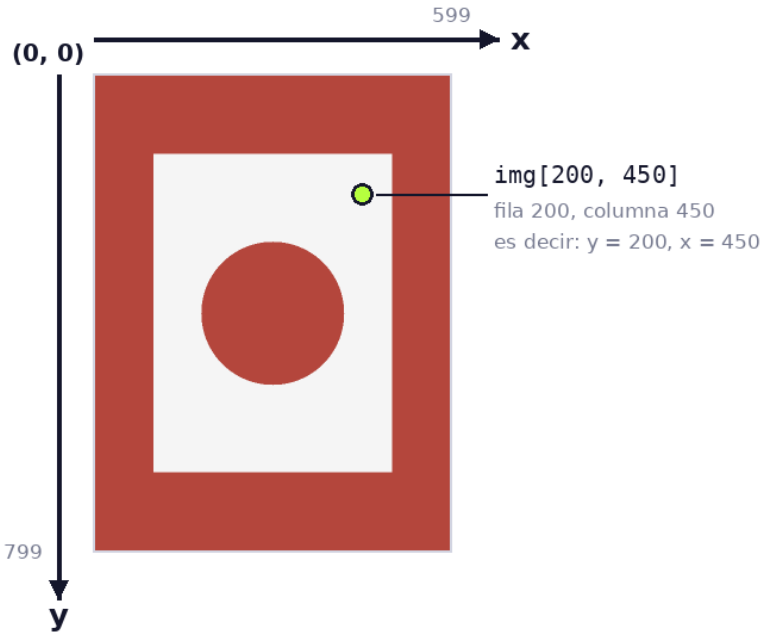
```
(800, 600, 3)
800 de alto
600 de ancho
```

- 800 es el alto: la cantidad de filas.
- 600 es el ancho: la cantidad de columnas.
- 3 son los canales: azul, verde y rojo. Tres números por píxel.

02 · COORDENADAS

Filas, columnas, y dónde está el (0, 0)

Mismo sistema que pygame: el origen arriba a la izquierda, la y hacia abajo.



- El (0, 0) es la esquina de arriba a la izquierda.
- La x crece hacia la derecha; la y crece hacia abajo.
- La última columna es la 599 y la última fila la 799: se cuenta desde 0.
- Para leer un píxel, OpenCV pide primero la fila y después la columna: img[y, x].

02 · DOS ÓRDENES

Dos órdenes que conviven

Hay funciones que piden (ancho, alto) y lugares que dan (alto, ancho). Hay que saber cuál es cuál.

Cuando le pasás un tamaño

```
# (ancho, alto)
set_mode((800, 600))
cv2.resize(img, (300, 400))
```

Las funciones que reciben un tamaño lo piden como ancho por alto, como se dice en voz alta.

Cuando leés la tabla

```
# (alto, ancho)
img.shape   # (800, 600, 3)
img[y, x]
```

La tabla se describe y se recorre como toda tabla: primero filas, después columnas.

Regla para recordar: si le pasás un tamaño a una función, va (ancho, alto). Si leés la tabla, va (fila, columna).

02 · SHAPE

Las medidas en dos variables

La línea que más se va a repetir en las tres clases.

medidas.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")
alto, ancho = img.shape[:2]

print("alto:", alto)
print("ancho:", ancho)
```

SALIDA

```
alto: 800
ancho: 600
```

- [:2] es slicing, como en las listas: se queda con los dos primeros valores del shape.
- Descarta el 3, que son los canales.
- alto, ancho = desarma esos dos valores en dos variables, en ese orden.
- Si escribís ancho, alto = …, las variables quedan cambiadas y nada avisa.

LA TRAMPA DE LA CLASE

OpenCV guarda los colores al revés: azul, verde, rojo.

Se llama BGR. Casi todo lo demás —pygame, Photoshop, la web— usa RGB: rojo, verde, azul.

02 · UN PÍXEL

¿Qué imprime?

El píxel de la esquina de retrato.png es un rojo ladrillo. Treinta segundos.

pixel.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

pixel = img[0, 0]
print(pixel)

b, g, r = pixel
print(f"azul={b} verde={g} rojo={r}")
```

SALIDA

?

- img[0, 0] es el píxel de la fila 0, columna 0: la esquina de arriba a la izquierda.
- La pregunta es en qué orden aparecen los tres números.
- Y cuál de los tres es el más grande.

02 · UN PÍXEL

Un píxel, y la trampa de la clase

Se indexa img[fila, columna]. Y lo que devuelve está en orden BGR.

pixel.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

pixel = img[0, 0]
print(pixel)

b, g, r = pixel
print(f"azul={b} verde={g} rojo={r}")
```

SALIDA

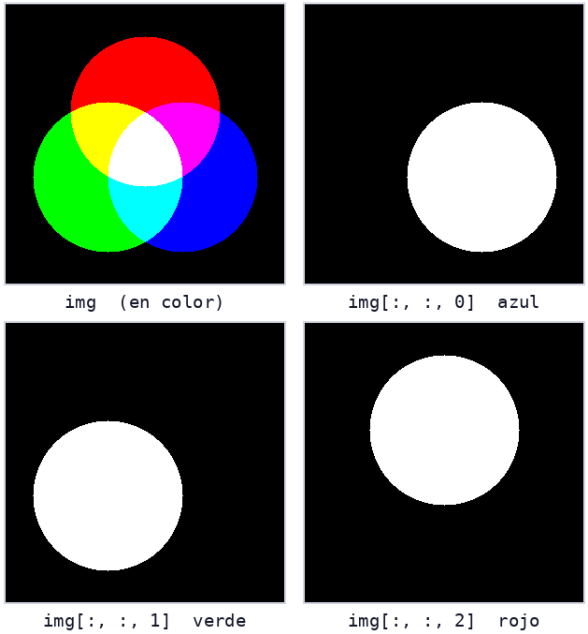
```
[ 60  70 180]
azul=60 verde=70 rojo=180
```

- El primero es el azul, el segundo el verde, el tercero el rojo.
- El 180 es el rojo: por eso el píxel es rojizo.
- b, g, r = pixel desarma los tres valores, igual que alto, ancho = … con el shape.

02 · CANALES

Los tres canales, por separado

Una imagen en color son tres tablas apiladas: una por color.



- Cada canal es una tabla de números de 0 a 255, como una foto en blanco y negro.
- Blanco significa mucho de ese color; negro, nada.
- img[:, :, 0] es el canal azul; el 1, el verde; el 2, el rojo.
- Ese orden —0 azul, 1 verde, 2 rojo— es exactamente lo que significa BGR.

02 · BGR

Algunos colores, en los dos órdenes

Para pasar un color de pygame a OpenCV, se da vuelta la tupla.

Color	pygame (RGB)	OpenCV (BGR)
negro	`(0, 0, 0)`	`(0, 0, 0)`
blanco	`(255, 255, 255)`	`(255, 255, 255)`
rojo	`(255, 0, 0)`	`(0, 0, 255)`
verde	`(0, 255, 0)`	`(0, 255, 0)`
azul	`(0, 0, 255)`	`(255, 0, 0)`
lima del curso	`(182, 255, 63)`	`(63, 255, 182)`

El negro, el blanco y el verde no cambian porque son simétricos: se leen igual al derecho y al revés.

02 · CUIDADO

El rojo en pygame y el rojo en OpenCV

Mismo color, dos órdenes distintos. Es el error que todos cometen la primera vez.

pygame · RGB

```
ROJO = (255, 0, 0)

pygame.draw.circle(
    pantalla, ROJO, p, 20)
```

Rojo, verde, azul. Como en el HTML y como en Photoshop.

OpenCV · BGR

```
ROJO = (0, 0, 255)

cv2.circle(
    img, p, 20, ROJO, -1)
```

Azul, verde, rojo. Dado vuelta.

Si dibujás algo y sale azul en vez de rojo, ya sabés qué pasó: tenés la tupla al revés.

02 · QUÉ ES

Qué es exactamente esa tabla

Tres preguntas que se le pueden hacer a cualquier imagen.

tipo.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

print(type(img))
print(img.dtype)
print(img.min(), img.max())
```

SALIDA

```
<class 'numpy.ndarray'>
uint8
60 245
```

- numpy.ndarray: una tabla de numpy. OpenCV usa numpy para guardar las imágenes.
- uint8: enteros sin signo de 8 bits. Quiere decir números enteros de 0 a 255, y nada más.
- min y max: el valor más chico y el más grande de toda la tabla.

03

Recortar

Y qué pasa cuando el archivo no está

03 · REPASO

Primero, el slicing de la clase 3

Recortar una imagen es exactamente esto, en dos dimensiones.

listas.py

```
colores = ["rojo", "verde", "azul", "lima", "negro"]

print(colores[1:3])
print(colores[:2])
print(colores[3:])
```

SALIDA

```
['verde', 'azul']
['rojo', 'verde']
['lima', 'negro']
```

- [1:3] va desde el índice 1, incluido, hasta el 3, sin incluirlo.
- Sin el primer número, arranca desde el principio. Sin el segundo, llega hasta el final.
- La cantidad de elementos es siempre hasta menos desde: 3 - 1 = 2.

03 · RECORTAR

En dos dimensiones: dos rangos

img[desde_y:hasta_y, desde_x:hasta_x]

- Antes de la coma, el rango de filas: de arriba hacia abajo.
- Después de la coma, el rango de columnas: de izquierda a derecha.
- Igual que en las listas: el desde se incluye, el hasta no.
- El alto del recorte es hasta_y - desde_y. El ancho, hasta_x - desde_x.

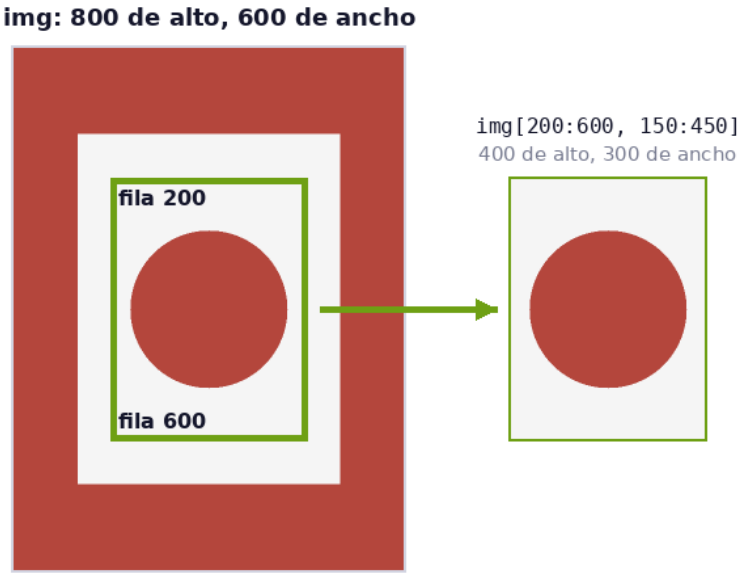
EL ORDEN, OTRA VEZ

Primero la y, después la x. Es el mismo orden del shape y de img[y, x]. Dentro de los corchetes, siempre filas primero.

03 · RECORTAR

img[200:600, 150:450], dibujado

El recuadro verde es lo que se queda. Todo lo demás se descarta.



- Filas de la 200 a la 600: 400 filas de alto.
- Columnas de la 150 a la 450: 300 columnas de ancho.
- El recorte es una imagen de 400 × 300, con su propio shape.
- Su (0, 0) es la esquina del recuadro, no la de la foto original.

03 · RECORTAR

Recortar en código

El mismo recorte de la imagen anterior, con el shape antes y después.

recortar.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")
print("entera: ", img.shape)

# [filas, columnas] = [desde_y:hasta_y,
#                      desde_x:hasta_x]
centro = img[200:600, 150:450]
print("recorte:", centro.shape)

cv2.imwrite("centro.png", centro)
```

SALIDA

```
entera:  (800, 600, 3)
recorte: (400, 300, 3)
```

- 600 - 200 = 400 de alto; 450 - 150 = 300 de ancho.
- El 3 de los canales no cambia: el recorte sigue siendo en color.
- imwrite guarda el recorte como centro.png. Lo vemos en unos minutos.

03 · BORDES

¿Y si me paso del borde?

La imagen tiene 800 filas. ¿Qué pasa si pido más?

borde.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

print(img[700:900, 0:10].shape)
print(img[900, 0])
```

SALIDA

```
(100, 10, 3)
Traceback (most recent call last):
IndexError: index 900 is out
of bounds for axis 0 with
size 800
```

- Un rango que se pasa del borde se corta solo: pidió 200 filas y dio 100. Sin avisar.
- Un índice suelto fuera de la tabla sí es error: IndexError, como en las listas.
- El primer caso es el peligroso: el recorte sale más chico y el programa sigue como si nada.

03 · CUIDADO

Un recorte no es una copia

Esto sorprende a todos. Se pinta de negro el recorte, y se mira la original.

vista.py

```
import cv2

img = cv2.imread("fotos/retrato.png")

esquina = img[0:10, 0:10]
esquina[:] = (0, 0, 0)

print(img[5, 5])
```

SALIDA

```
[0 0 0]
```

- Pintar el recorte pintó también la imagen original.
- El recorte no es una tabla nueva: es una ventana que mira una parte de la misma tabla.
- esquina[:] = (0, 0, 0) significa: todo lo de adentro pasa a valer (0, 0, 0).

03 · CUIDADO

Si querés una copia, pedila

Lo mismo, con .copy() al final del recorte.

copia.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

esquina = img[0:10, 0:10].copy()
esquina[:] = (0, 0, 0)

print(img[5, 5])
```

SALIDA

```
[ 60  70 180]
```

- Con .copy(), el recorte es una tabla nueva e independiente.
- Pintarlo ya no toca la original: el píxel sigue siendo el rojo ladrillo.
- Regla: si vas a modificar un recorte y querés conservar la original, .copy().

03 · GUARDAR

imwrite: guardar en el disco

imwrite viene de image write: escribir una imagen.

guardar.py

```
import cv2

img = cv2.imread("fotos/retrato.png")
centro = img[200:600, 150:450]

ok = cv2.imwrite("centro.png", centro)
print(ok)
```

SALIDA

```
True
```

- Primero la ruta donde se va a guardar; después la imagen.
- La extensión decide el formato: .png o .jpg.
- Devuelve True si pudo guardar.
- Si ya existe un archivo con ese nombre, lo reemplaza sin preguntar.

03 · GUARDAR

Cuando imwrite no puede guardar

Lo mismo, pero guardando adentro de una carpeta salida/ que todavía no existe.

falla_guardar.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")

ok = cv2.imwrite("salida/centro.png", img)
print(ok)
```

SALIDA

```
False
```

- Si la carpeta no existe, no guarda nada.
- Y no da error: devuelve False, en silencio.
- Por eso el práctico crea la carpeta antes de guardar, con os.makedirs. Lo vemos en la sección 06.

REGLA

imread e imwrite no avisan cuando fallan. Hay que preguntarles.

imread devuelve None; imwrite devuelve False. Ninguno corta el programa. El error aparece después, lejos de donde pasó.

03 · EL ERROR

Cuando el archivo no está

imread no explota. Hace algo peor: sigue como si nada.

falla.py

```python
import cv2

img = cv2.imread("fotos/no_existe.png")

print(img)
print(img is None)
```

SALIDA

```
[ WARN:0 ] findDecoder imread_:
can't open/read file: check
file path/integrity

None
True
```

- Devuelve None y escribe un aviso, pero no corta el programa.
- El aviso dice exactamente qué archivo no pudo abrir. Hay que leerlo.
- is None pregunta si la variable está vacía. Es la forma correcta de preguntarlo.

03 · EL ERROR

Dónde explota de verdad

El archivo es retrato.png. Acá se escribió .jpg: un error de tres letras.

explota.py

```python
import cv2

img = cv2.imread("fotos/retrato.jpg")
alto, ancho = img.shape[:2]
```

SALIDA

```
[ WARN:0 ] can't open/read file
Traceback (most recent call last):
AttributeError: 'NoneType'
object has no attribute
'shape'
```

- El programa no corta en el imread: corta una línea después.
- El error dice NoneType: la variable img vale None, y None no tiene shape.
- Con veinte líneas de por medio, el error aparece lejos de la causa. Por eso cuesta encontrarlo.

03 · LA SOLUCIÓN

Revisar antes de usar

Una función que abre la imagen y avisa en el momento si no pudo.

cargar.py

```python
import cv2

def cargar(ruta):
    """Devuelve la imagen, o None con aviso."""
    img = cv2.imread(ruta)
    if img is None:
        print(f"No pude abrir: {ruta}")
        return None
    return img


img = cargar("fotos/retrato.png")
if img is not None:
    print("abierta:", img.shape)
```

SALIDA

```
abierta: (800, 600, 3)
```

- Es el primer manejo de errores del curso, y no necesita try.
- La función devuelve None a propósito: el que la llama decide qué hacer.
- Esta función se copia al práctico y al Obligatorio tal cual está.

03 · RUTAS

El error más común de hoy: desde dónde se ejecuta

"fotos/retrato.png" es una ruta relativa. Relativa a qué es la pregunta.

- Una ruta relativa se busca a partir de la carpeta desde donde se ejecuta Python.
- No a partir de la carpeta donde está guardado el archivo .py.
- Si en VS Code abriste la carpeta ejercicios-clase y apretás ▷, Python busca ejercicios-clase/fotos, no clase-09/fotos.
- Dos soluciones: abrir clase-09 como carpeta en VS Code, o en la terminal escribir cd clase-09 y después python main.py.

CÓMO DARSE CUENTA

Si el aviso dice can't open/read file y el archivo está ahí, a la vista, es esto. La foto existe, pero Python la está buscando en otra carpeta.

04

Transformar

Gris, tamaño y espejo

04 · GRIS

cvtColor: cambiar la forma de los colores

cvtColor viene de convert color: convertir el color.

gris.py

```
import cv2

img = cv2.imread("fotos/retrato.png")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("color:", img.shape)
print("gris: ", gris.shape)
print("un pixel:", gris[0, 0])
```

- Recibe la imagen y la conversión que se quiere hacer.
- COLOR_BGR2GRAY significa de BGR a gris. El 2 se lee to, en inglés: de uno a otro.
- Devuelve una imagen nueva. La original, img, no cambia.
- Casi todo lo de las clases 10 y 11 empieza con esta línea.

04 · GRIS

Pasar a gris, y qué cambia

Lo que cambia se ve en el shape y en el píxel.

gris.py

```python
import cv2

img = cv2.imread("fotos/retrato.png")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("color:", img.shape)
print("gris: ", gris.shape)
print("un pixel:", gris[0, 0])
```

SALIDA

```
color: (800, 600, 3)
gris:  (800, 600)
un pixel: 102
```

- El shape pierde el 3: de tres números por píxel pasa a uno solo.
- Ese número es el brillo: 0 es negro, 255 es blanco.
- El píxel que en color era [60, 70, 180], en gris vale 102. La próxima diapositiva dice por qué.

04 · GRIS

De dónde sale el 102

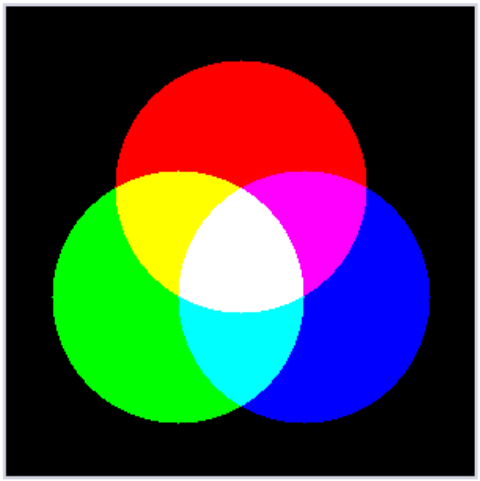
El gris no es el promedio de los tres. Cada color pesa distinto.

1. La fórmula es: 0,299 × rojo + 0,587 × verde + 0,114 × azul.
2. Con el píxel [60, 70, 180]: el rojo es 180, el verde 70, el azul 60.
3. 0,299 × 180 + 0,587 × 70 + 0,114 × 60 = 53,82 + 41,09 + 6,84.
4. Suma 101,75, y se redondea a 102.
5. Es la misma fórmula de luminosidad del ejercicio de la clase 2.

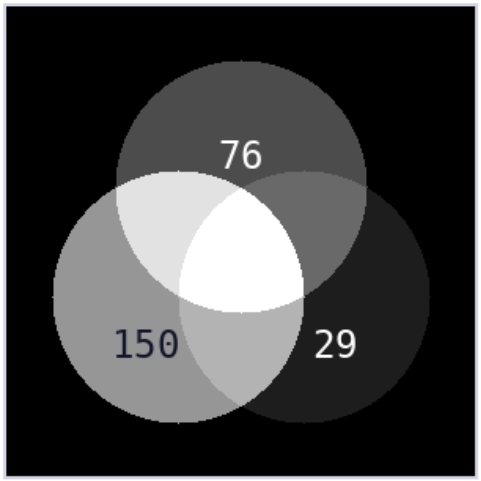
El promedio daría (180 + 70 + 60) ÷ 3 = 103,3. Parecido, pero no igual: en otros colores la diferencia es enorme.

El mismo 255 no pesa lo mismo

Tres círculos puros: cada uno tiene 255 en su canal y 0 en los otros dos.



en color



en gris

- El rojo puro queda en 76; el verde puro, en 150; el azul puro, en 29.
- El verde queda cinco veces más claro que el azul, aunque los dos tenían 255.
- Así percibe el ojo humano el brillo: es mucho más sensible al verde.
- La fórmula copia al ojo. Por eso los pesos son tan distintos.

04 · TAMAÑO

resize: cambiar el tamaño

resize: cambiar de tamaño.

miniatura.py

```
import cv2

img = cv2.imread("fotos/serie_azul.png")
alto, ancho = img.shape[:2]

nuevo_ancho = 300
nuevo_alto = int(alto * nuevo_ancho / ancho)

chica = cv2.resize(img, (nuevo_ancho, nuevo_alto))

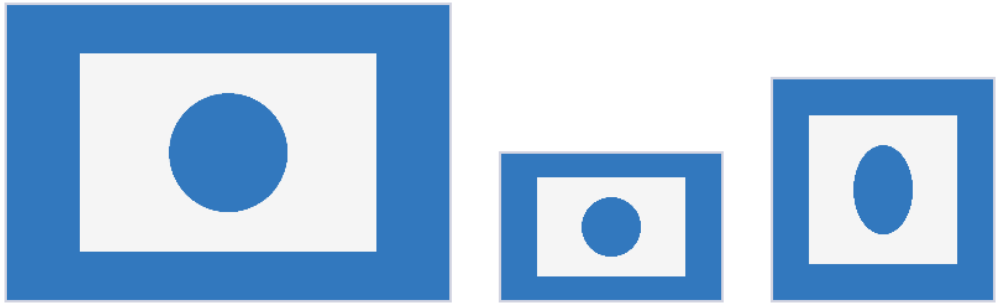
print("original: ", img.shape)
print("miniatura:", chica.shape)
```

- Recibe la imagen y el tamaño nuevo.
- El tamaño va como (ancho, alto): al revés del shape.
- Si el tamaño nuevo no respeta la proporción de la original, la imagen se deforma.
- Las líneas 6 y 7 calculan el alto justo para que no se deforme.

04 · TAMAÑO

Proporcional contra deformada

La misma imagen de 1200 × 800, achicada de dos maneras.



original
1200 × 800

proporcional
300 × 200

deformada
300 × 300

- La proporcional mantiene la relación entre ancho y alto: 3 a 2.
- La deformada se forzó a un cuadrado de 300 × 300.
- Fijate en el círculo: en la deformada es un óvalo.
- Con fotos de personas, la deformación se nota todavía más.

04 · TAMAÑO

La regla de tres, paso a paso

Cómo se calcula el alto para que la imagen no se deforme.

1. La original mide 1200 de ancho y 800 de alto.
2. Queremos 300 de ancho: es la cuarta parte de 1200.
3. El alto también tiene que ser la cuarta parte: 800 × 300 ÷ 1200 = 200.
4. En código: int(alto * nuevo_ancho / ancho).
5. El int() porque resize solo acepta medidas enteras: no existe medio píxel.

Primero se multiplica y después se divide. Con la división entera // el orden importaría: 800 // 1200 da 0.

04 · TAMAÑO

Achicar sin deformar

El cálculo de la diapositiva anterior, en código.

miniatura.py

```python
import cv2

img = cv2.imread("fotos/serie_azul.png")
alto, ancho = img.shape[:2]

nuevo_ancho = 300
nuevo_alto = int(alto * nuevo_ancho / ancho)

chica = cv2.resize(img, (nuevo_ancho, nuevo_alto))

print("original: ", img.shape)
print("miniatura:", chica.shape)
```

SALIDA

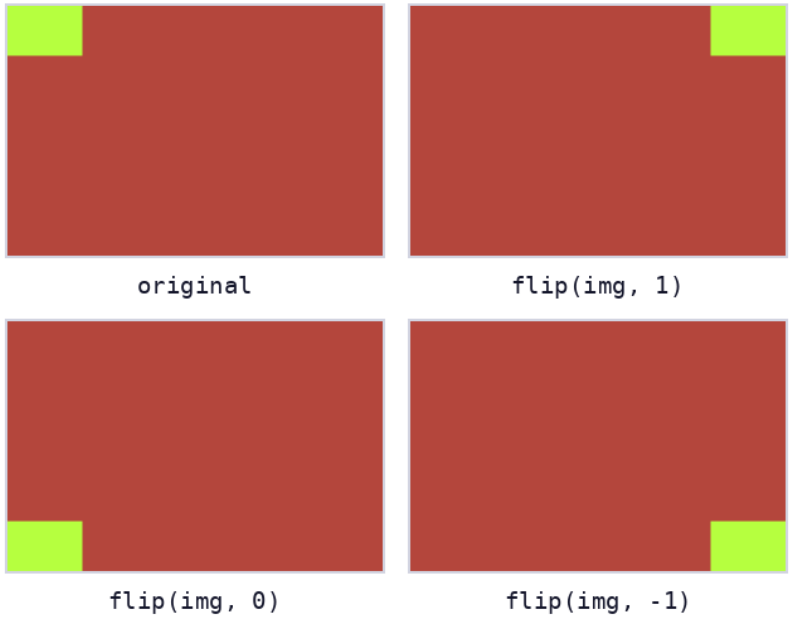
```
original:  (800, 1200, 3)
miniatura: (200, 300, 3)
```

- img.shape[:2] da alto y ancho, sin los canales.
- nuevo_alto queda en 200: la regla de tres.
- En el resize va (nuevo_ancho, nuevo_alto): ancho primero.

04 · ESPEJO

flip: los tres espejos

flip: dar vuelta. Un solo número decide el eje.



- flip(img, 1) espeja en horizontal: lo de la izquierda pasa a la derecha.
- flip(img, 0) espeja en vertical: lo de arriba pasa abajo.
- flip(img, -1) hace los dos a la vez.
- El cuadrado lima marca dónde terminó la esquina de arriba a la izquierda.

04 · ESPEJO

Espejar, en números

El 1 es el que van a usar la clase que viene, con la cámara.

espejo.py

```python
import cv2

img = cv2.imread("fotos/marca.png")
espejo = cv2.flip(img, 1)

print("orig   izq:", img[40, 20])
print("orig   der:", img[40, 580])
print("espejo izq:", espejo[40, 20])
print("espejo der:", espejo[40, 580])
```

SALIDA

```
orig   izq: [ 63 255 182]
orig   der: [ 60  70 180]
espejo izq: [ 60  70 180]
espejo der: [ 63 255 182]
```

- El lima estaba a la izquierda y después del flip está a la derecha.
- Con la cámara se usa siempre: sin espejar, levantar la mano derecha se ve como la izquierda.
- flip también devuelve una imagen nueva: la original no cambia.

04 · CHULETA

Todo lo de hasta acá, en una tabla

Con esto se resuelve el práctico entero.

Línea	Qué hace
`cv2.imread(ruta)`	abre una imagen, o devuelve None
`cv2.imwrite(ruta, img)`	la guarda; devuelve True o False
`img.shape`	(alto, ancho, canales)
`img[y, x]`	el píxel de esa fila y columna
`img[y1:y2, x1:x2]`	recorta (no es una copia)
`cv2.cvtColor(img, ...)`	convierte los colores, por ejemplo a gris
`cv2.resize(img, (an, al))`	cambia el tamaño
`cv2.flip(img, 1)`	espeja

El resto está en docs.opencv.org. Buscar ahí también cuenta como programar.

05

Dibujar encima

Los draw de pygame, con otro nombre

05 · DIBUJAR

Dibujar cambia la imagen

A diferencia de cvtColor, resize y flip, las funciones de dibujo no devuelven una imagen nueva.

- rectangle, circle, line y putText dibujan sobre la imagen que reciben.
- No devuelven una imagen nueva: modifican esa misma.
- Si querés conservar la original sin dibujos, dibujá sobre img.copy().
- El color va en BGR, como todo en OpenCV.

POR QUÉ IMPORTA

Si dibujás un recuadro sobre la imagen y después la pasás a gris o la guardás, el recuadro va incluido. Ya forma parte de la tabla de números.

05 · DIBUJAR

rectangle: un rectángulo

Cinco argumentos, siempre en este orden.

marcar.py

```
import cv2

img = cv2.imread("fotos/retrato.png")
LIMA = (63, 255, 182)   # BGR

# esquina sup. izq. y esquina inf. der.
cv2.rectangle(img, (150, 200), (450, 600),
              LIMA, 3)

cv2.circle(img, (300, 400), 40, LIMA, -1)

cv2.putText(img, "Retrato", (150, 180),
            cv2.FONT_HERSHEY_SIMPLEX,
            1.0, LIMA, 2)

cv2.imwrite("marcada.png", img)
```

- La imagen sobre la que se dibuja.
- La esquina de arriba a la izquierda, como (x, y).
- La esquina de abajo a la derecha, como (x, y).
- El color, en BGR.
- El grosor del borde en píxeles. Con -1, el rectángulo sale relleno.

05 · DIBUJAR

circle: un círculo

Mismo esquema: imagen, dónde, cuánto, color, grosor.

marcar.py

```
import cv2

img = cv2.imread("fotos/retrato.png")
LIMA = (63, 255, 182)   # BGR

# esquina sup. izq. y esquina inf. der.
cv2.rectangle(img, (150, 200), (450, 600),
              LIMA, 3)

cv2.circle(img, (300, 400), 40, LIMA, -1)

cv2.putText(img, "Retrato", (150, 180),
            cv2.FONT_HERSHEY_SIMPLEX,
            1.0, LIMA, 2)

cv2.imwrite("marcada.png", img)
```

- El centro, como (x, y).
- El radio en píxeles: 40 de radio son 80 de diámetro.
- El color, en BGR.
- -1 lo rellena. Con un número positivo, solo se dibuja el borde.

putText: escribir

El que más argumentos tiene. Van de a uno.

marcar.py

```
import cv2

img = cv2.imread("fotos/retrato.png")
LIMA = (63, 255, 182)   # BGR

# esquina sup. izq. y esquina inf. der.
cv2.rectangle(img, (150, 200), (450, 600),
              LIMA, 3)

cv2.circle(img, (300, 400), 40, LIMA, -1)

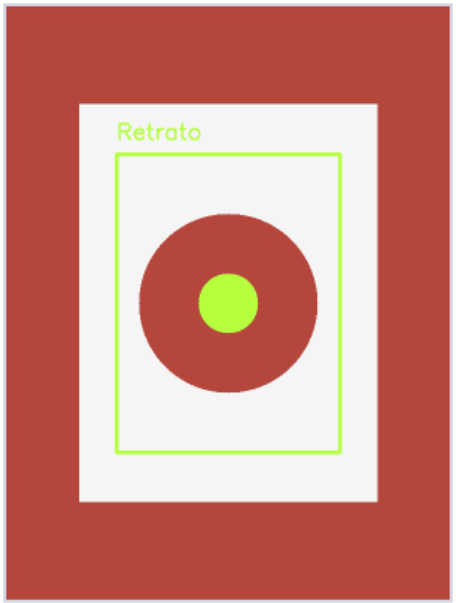
cv2.putText(img, "Retrato", (150, 180),
            cv2.FONT_HERSHEY_SIMPLEX,
            1.0, LIMA, 2)

cv2.imwrite("marcada.png", img)
```

- La imagen y el texto, entre comillas.
- El punto donde arranca, como (x, y). Ojo: es la esquina de abajo a la izquierda del texto, no la de arriba.
- La fuente: FONT_HERSHEY_SIMPLEX es la más legible de las que trae OpenCV.
- La escala: 1.0 es el tamaño base; 2.0, el doble.
- El color y el grosor de la letra.

El resultado

Lo que queda guardado en marcada.png después de las tres funciones.



- El recuadro lima va de (150, 200) a (450, 600), con 3 píxeles de borde.
- El círculo relleno, con centro en (300, 400) y 40 de radio.
- El texto arranca en (150, 180): su base queda 20 píxeles arriba del recuadro.
- Todo se dibujó sobre img: en la variable ya no queda la imagen sin dibujos.

05 · CUIDADO

OpenCV no escribe tildes

Las fuentes de putText no tienen acentos ni eñe.



sin tildes: bien

con tildes: ??

- Cada letra que no conoce sale como dos signos de pregunta.
- Pasa con á, é, í, ó, ú, ñ y ü.
- En putText, escribí sin tildes: Titulo, Cancion.
- Para textos con tildes, la clase 11 dibuja el texto con pygame, que sí las tiene.

05 · PARALELO

Es lo mismo que ya saben

Las dos bibliotecas dibujan igual. Cambian los nombres y el orden de los argumentos.

pygame

```
pygame.draw.circle(
    pantalla, LIMA,
    (300, 400), 40)
```

Primero dónde, después el color, después la figura.

OpenCV

```
cv2.circle(
    img, (300, 400), 40,
    LIMA, -1)
```

El color va casi al final, y el grosor -1 es para rellenar.

Nadie se acuerda del orden de memoria. Se mira la documentación, y está bien que así sea.

05 · PEGAR

Pegar una imagen en otra: el lienzo

Para pegar hace falta algo donde pegar: un lienzo vacío.

pegar.py

```
import cv2, numpy as np

foto = cv2.imread("fotos/retrato.png")
mini = cv2.resize(foto, (150, 200))

# un lienzo blanco: 300 de alto, 400 de ancho
lienzo = np.full((300, 400, 3), 255, np.uint8)

# la esquina de la mini va en x=20, y=50
lienzo[50:250, 20:170] = mini

print("lienzo:", lienzo.shape)
print("mini:  ", mini.shape)
cv2.imwrite("pegada.png", lienzo)
```

- np es numpy, importado con un nombre corto: import numpy as np.
- np.full crea una tabla nueva del tamaño pedido, llena de un mismo valor.
- (300, 400, 3) es el tamaño, en el orden del shape: alto, ancho, canales.
- 255 es el valor de relleno: blanco. np.uint8, el tipo: enteros de 0 a 255.

05 · PEGAR

Pegar: slicing del lado izquierdo

El slicing de siempre, pero a la izquierda del igual: en vez de leer, escribe.

pegar.py

```
import cv2, numpy as np

foto = cv2.imread("fotos/retrato.png")
mini = cv2.resize(foto, (150, 200))

# un lienzo blanco: 300 de alto, 400 de ancho
lienzo = np.full((300, 400, 3), 255, np.uint8)

# la esquina de la mini va en x=20, y=50
lienzo[50:250, 20:170] = mini

print("lienzo:", lienzo.shape)
print("mini:  ", mini.shape)
cv2.imwrite("pegada.png", lienzo)
```

- lienzo[50:250, 20:170] es un hueco de 200 filas por 150 columnas.
- = mini copia la mini adentro de ese hueco.
- El hueco tiene que medir exactamente lo mismo que la mini: 200 de alto por 150 de ancho.
- La esquina de arriba a la izquierda de la mini queda en x = 20, y = 50.

05 · PEGAR

Pegar, completo

El programa entero, con los dos shapes para comprobar que coinciden.

pegar.py

```python
import cv2, numpy as np

foto = cv2.imread("fotos/retrato.png")
mini = cv2.resize(foto, (150, 200))

# un lienzo blanco: 300 de alto, 400 de ancho
lienzo = np.full((300, 400, 3), 255, np.uint8)

# la esquina de la mini va en x=20, y=50
lienzo[50:250, 20:170] = mini

print("lienzo:", lienzo.shape)
print("mini:  ", mini.shape)
cv2.imwrite("pegada.png", lienzo)
```

SALIDA

```
lienzo: (300, 400, 3)
mini:   (200, 150, 3)
```

- La mini mide 200 × 150: el resize se pidió como (150, 200), ancho primero.
- Si el hueco no mide lo mismo, numpy avisa: could not broadcast input array from shape…
- Ese mensaje siempre significa lo mismo: los tamaños no coinciden.

05 · PEGAR

Cómo queda

El lienzo blanco, con la mini pegada en su lugar.



- Se usa para armar collages, marcos y hojas de contacto: los ejercicios 9.2 y 9.3.
- Y en la clase 11, para poner la imagen de la cámara adentro de una composición.

06

Una carpeta entera

El for que hace el trabajo de una tarde

06 · OS

El módulo os

os viene de operating system: sistema operativo. Viene con Python, no hay que instalarlo.

- Sirve para hablar con el sistema de archivos: carpetas, nombres y rutas.
- os.listdir: la lista de lo que hay adentro de una carpeta.
- os.path.join: arma una ruta juntando una carpeta y un nombre.
- os.makedirs: crea una carpeta.

POR QUÉ HACE FALTA

Sin os, habría que escribir a mano el nombre de cada foto. Con os, el programa descubre solo qué fotos hay, y funciona igual con 4 o con 400.

06 · LISTAR

listdir, sin ordenar

Lo que devuelve listdir tal cual, sin tocar nada.

listar.py

```
import os

print(os.listdir("fotos"))
```

SALIDA

```
['serie_azul.png', 'marca.png',
 'retrato.png', 'bocetos.png']
```

- Devuelve una lista de nombres, sin la carpeta adelante.
- El orden no es alfabético: depende del sistema. En tu computadora puede salir otro.
- Por eso siempre se usa sorted(): para que el orden sea el mismo en cualquier máquina.

06 · LISTAR

Ordenar y quedarse con las imágenes

Dos arreglos: sorted para el orden, y un filtro para la extensión.

listar.py

```
import os

archivos = os.listdir("fotos")

imagenes = [a for a in sorted(archivos)
            if a.endswith(".png")]

for nombre in imagenes:
    print(nombre)
```

SALIDA

```
bocetos.png
marca.png
retrato.png
serie_azul.png
```

- listdir devuelve todo lo que hay: imágenes, archivos sueltos y carpetas.
- endswith(".png") pregunta si el nombre termina así. Sin el filtro, imread fallaría con el primer .txt.
- La lista se arma con un for adentro de corchetes: una forma corta del for con append.

06 · RECORRER

os.path.join: armar la ruta

listdir devolvió solo el nombre. Para abrir el archivo hace falta la ruta completa.

recorrer.py

```
import cv2, os

imagenes = [a for a in sorted(os.listdir("fotos"))
            if a.endswith(".png")]

for nombre in imagenes:
    ruta = os.path.join("fotos", nombre)
    img = cv2.imread(ruta)
    if img is None:
        continue
    alto, ancho = img.shape[:2]
    print(f"{nombre:16} {ancho} x {alto}")
```

- os.path.join("fotos", "retrato.png") da "fotos/retrato.png".
- Usa la barra que corresponde a cada sistema: / en Mac y Linux, \ en Windows.
- Escribir "fotos/" + nombre también suele funcionar, pero join es lo correcto en cualquier máquina.

06 · RECORRER

continue: saltear y seguir

Si una imagen no se puede abrir, el programa no se tiene que caer.

recorrer.py

```
import cv2, os

imagenes = [a for a in sorted(os.listdir("fotos"))
            if a.endswith(".png")]

for nombre in imagenes:
    ruta = os.path.join("fotos", nombre)
    img = cv2.imread(ruta)
    if img is None:
        continue
    alto, ancho = img.shape[:2]
    print(f"{nombre:16} {ancho} x {alto}")
```

- if img is None pregunta si imread falló.
- continue saltea el resto de esta vuelta del for y pasa al archivo siguiente.
- Sin el continue, la línea del shape explotaría con el error de NoneType.
- Es el mismo continue de los bucles de la clase 3.

06 · RECORRER

Procesarlas todas

Acá se juntan el for de la clase 3 y todo lo de hoy.

recorrer.py

```python
import cv2, os

imagenes = [a for a in sorted(os.listdir("fotos"))
            if a.endswith(".png")]

for nombre in imagenes:
    ruta = os.path.join("fotos", nombre)
    img = cv2.imread(ruta)
    if img is None:
        continue
    alto, ancho = img.shape[:2]
    print(f"{nombre:16} {ancho} x {alto}")
```

SALIDA

```
bocetos.png      900 x 900
marca.png        600 x 400
retrato.png      600 x 800
serie_azul.png   1200 x 800
```

- {nombre:16} reserva 16 caracteres para el nombre: es el formateo de la clase 2.
- Así las medidas quedan alineadas en columna.
- Este recorrido es el esqueleto del práctico: solo falta qué hacer con cada imagen.

06 · CARPETA

makedirs: crear la carpeta de salida

Lo que hacía falta para que imwrite no devolviera False.

crear.py

```
import os

os.makedirs("salida", exist_ok=True)
print(os.path.isdir("salida"))
```

SALIDA

```
True
```

- Crea la carpeta salida, al lado del programa.
- exist_ok=True: si la carpeta ya existe, no hace nada y no da error.
- Sin eso, la segunda vez que corrés el programa da FileExistsError: la carpeta ya está.

06 · RESUMEN

Lo que tienen que llevarse de hoy

Seis ideas. Si las seis están claras, el práctico sale.

- Una imagen es una tabla de números: shape da (alto, ancho, canales).
- Un píxel se pide como img[y, x], y sus colores vienen en orden BGR.
- Recortar es slicing, y el recorte no es una copia.
- imread e imwrite no avisan cuando fallan: se revisa None y False.
- resize pide (ancho, alto); el alto se calcula con regla de tres.
- os.listdir, sorted, os.path.join y continue recorren una carpeta entera.

LO QUE SIGUE

El práctico: el procesador de obras. Usa todo esto junto, y lo que escriban hoy en filtros.py se usa sobre la cámara la clase que viene.

07 · PRÁCTICO

El práctico va en otro archivo

A partir de acá se proyecta clase9_practico.pptx.

- Ahí está la consigna del práctico y el código de arranque para copiar.
- Y los tres ejercicios de clase, 9.1 a 9.3, en orden de prioridad.
- Las soluciones van en un tercer archivo, clase9_soluciones.pptx.
- Ese último se comparte después de que lo hayan intentado.

DÓNDE VA EL CÓDIGO

En el repositorio ejercicios-clase, carpeta clase-09, un archivo .py por ejercicio y un commit por ejercicio. Y push antes de irse.

PARA LA PRÓXIMA CLASE

La cámara en vivo

La misma imagen de hoy, pero treinta veces por segundo.

Clase 10 · viernes 23/10. Vengan con el stack instalado y la cámara andando.